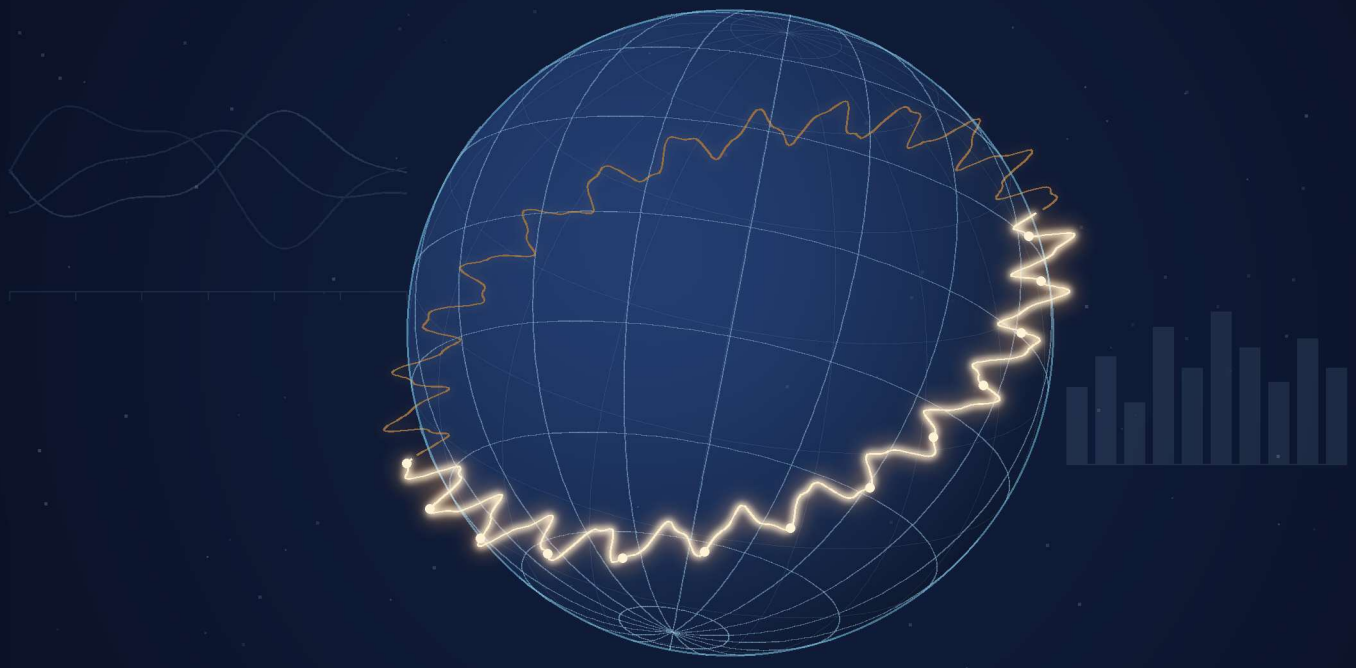
R for Earth and Sustainability

— DATA ANALYTICS —

A Beginner's Guide to Earth Data Analytics
with GIS, Mapping, Time Series, and Machine Learning in R

No Prior Coding Experience Required



Learn R · Explore Earth

Antonios E. Marsellos, Ph.D. · Katerina G. Tsakiri, Ph.D.

R for Earth and Sustainability Data Analytics

Maps show where.
Time series show how.
Data analytics helps us understand why.

R for Earth and Sustainability Data Analytics

A Beginner's Guide to Earth Data Analytics with GIS, Mapping, Time Series, and Machine Learning in R

Antonios E. Marsellos, Ph.D.
Katerina G. Tsakiri, Ph.D.

Earth Data Analytics Press

2026

ISBN: 979-8-9965635-0-0 (Paperback)
ISBN: 979-8-9965635-1-7 (eBook)
ISBN: 979-8-9965635-2-4 (Hardcover)

Library of Congress Control Number: 2026915118

Publisher Earth Data Analytics Press
Queens, New York

Authors
Antonios E. Marsellos, Ph.D.
Department of Geology, Environment, and Sustainability
Hofstra University

Katerina G. Tsakiri, Ph.D.
Department of Information Systems, Analytics and Supply Chain Management
Rider University

Cover Design
Cover design by the authors.

First Edition
First published in 2026

Contact
For questions, feedback, or educational inquiries, please contact the authors.

Disclosure: The authors are co-founders and owners of ResForMe LLC (resforme.com), a company that provides research mentorship services to high school students. As owners of the company, the authors may benefit financially from references to ResForMe contained in this book. Any discussion of ResForMe is intended solely for educational and informational purposes and does not imply endorsement by the authors' affiliated universities or institutions.

The views expressed in this publication are those of the authors and do not necessarily reflect the views of their affiliated institutions.

Brief Contents

Table of contents

Preface

Why Coding Matters in Earth and Sustainability Data Analytics

As more fields rely on data, the ability to analyze, interpret, and communicate information effectively is essential across disciplines. When we study data tied to places (geospatial analysis), environmental science, or changes over time (time series modeling), there are two main ways to work with data: graphical software and coding. Both have value. The difference is how far they can take you.

Coding lets you do what graphical interfaces cannot.

Graphical software is genuinely useful. It is fast, visual, and practical. Imagine being in the field collecting data on your phone or tablet. You need to see a map, click a few buttons, and make a quick decision. A graphical interface is perfect for that moment. It is also the easiest way to introduce GIS or time series analysis to beginners.

But graphical tools are built for convenience. They are not built for full control. Menus change, buttons move, and workflows evolve. A process you learned today may look different in a few months. Repeating the exact same analysis later can become difficult.

Coding works differently. It is stable. Every step is written, visible, and repeatable. If you run the same code tomorrow, next year, or on another computer, you get the same result. Instead of depending on where a button is located, you are defining the entire process yourself.

From Clicking to Thinking

Transparency is one of the most powerful advantages of coding. You can see every step, and changing one line immediately shows its effect.

Think of a simple example. You process satellite data and realize that one parameter needs adjustment. In a graphical tool, you may need to repeat the entire workflow step by step. This can take hours. In coding, you change one value and run the script again. The result updates in seconds.

This changes how you work. Instead of repeating steps, you refine ideas. In graphical workflows, improvement often means repeating the process; in coding workflows, it happens instantly. This speed encourages exploration: you can try more ideas, test more hypotheses, and reach better conclusions.

Coding Inside vs Outside Software

Some graphical tools offer scripting options. SPSS, GIS platforms, and other analytical software allow users to write code within their systems. At first glance, this seems like the best of both worlds. You get the convenience of a graphical interface with the power of coding, and you can automate tasks while still using familiar tools. In practice, it often creates new challenges.

These scripting environments are usually limited, harder to debug, and less flexible. They depend on proprietary systems and smaller user communities. When something goes wrong, finding help can be difficult. Being outside of a specific software environment, using languages like R or Python, provides more freedom. You can use any free add-on or tool that the worldwide community of users has shared (these free add-ons are called libraries), without being tied to a specific platform's limitations. When you encounter an issue, you can search for solutions across a vast ecosystem of users and resources. This makes learning and troubleshooting much easier.

R or Python?

Millions of users work with these languages. There are tutorials, forums, examples, and shared solutions for almost every problem. If you search for an issue, chances are someone has already solved it.

Even generative AI tools perform better with these languages. They have been trained on large amounts of public code, which makes their suggestions more accurate and more useful.

Choosing a language is not only about how you type commands (its syntax). It is about choosing a community and a support system you can rely on for years.

Why R

Tools like Excel, SPSS, and Tableau are useful in specific contexts. Excel is simple and familiar. SPSS provides structured statistical workflows. Tableau creates appealing visualizations. However, they often separate the workflow. You may clean data in one tool, analyze it in another, and visualize it somewhere else.

R brings everything together. You can import data, clean it, analyze it, model it, and visualize it in one place. You can also integrate geospatial analysis directly into the same workflow.

Historically, GIS began with coding. Early systems such as GRASS GIS relied on command line tools in UNIX environments. Later, graphical systems like ArcView made GIS more accessible. Today, tools like ArcGIS Pro and QGIS are widely used.

R reconnects GIS with its computational roots while keeping modern flexibility. It allows you to design your entire workflow from start to finish.

Learning One Language Helps You Learn Others

A common question is whether to learn R or Python. R is strong in statistics, visualization, and research workflows. Python is broader and widely used in software development and machine learning.

Both are powerful. More importantly, both teach you how to think computationally: how to break a problem into clear, repeatable steps a computer can follow. The real goal is not choosing a language but learning how to design solutions. Learning one language does not limit you. It gives you a foundation to learn others. The skills you develop in R will transfer to Python, and vice versa.

The New Reality: Coding with Generative AI

A major shift has occurred in recent years. Students no longer need to understand every line of code to begin producing meaningful results. With generative AI, a user can write or generate advanced scripts, run them, and immediately see outputs. This creates an important contrast. In graphical software, even simple workflows require time. Each step must be performed manually. There is no way to compress dozens of clicks into a few seconds.

In coding, a full workflow can run instantly. You can generate a script, execute it, observe the result, and refine it in a continuous loop. What once took hours or days can now take minutes.

More importantly, coding in environments such as R allows users to immediately access advanced analytical tools. Within seconds, a student can move from raw data to visualization, geospatial mapping, and time series analysis, and even apply advanced prediction methods (with names like neural networks and random forests) that you will meet, step by step, later in the book. These are techniques that are widely used in industry and often associated with high-value analytical work.

Seeing meaningful results early changes motivation. Instead of spending weeks learning software mechanics before producing useful output, students can explore the entire data analysis workflow from the beginning. This immediate feedback encourages curiosity, confidence, and a stronger desire to learn.

Understanding still matters. Over time, users should learn what their code does and how each step works. However, the entry point is no longer a barrier. Students can begin by exploring results and gradually build deeper understanding as they refine their workflows. This reflects how many professionals work: they start with a question, investigate it with code, and learn the details along the way.

This is a fundamental shift in how we teach data analytics. It allows us to focus on creativity, exploration, and problem-solving from the start rather than getting stuck in mechanics. This is an unusually accessible time to begin research and data analysis because advanced computational tools are more widely available than ever before.

A Balanced Perspective

Graphical tools and coding do not compete. They complement each other. A graphical interface is like using a prepared meal kit. It is fast and helpful when you need something ready. Coding is like being a chef. You control the ingredients, the process, and the final presentation. Each approach has its place. But only coding allows unlimited creativity.

Final Thought

A graphical interface helps you complete a task. Coding allows you to design the entire workflow. It gives you the power to create solutions, not just use them. In Earth and Sustainability Data Analytics, this means you can ask new questions, explore environmental and societal systems, and develop new analytical workflows for real-world problems. You are not limited by what a software company has built; you can build your own tools. This book is an invitation to move from using tools to creating solutions.

About This Book

R as a Tool

Earth data are everywhere. From satellites and weather stations to groundwater sensors, climate records, environmental monitoring networks, and GPS systems, the Earth continuously generates information about how our planet is changing. The challenge is not access to data; it is knowing how to understand it.

This book was written to help students take their first steps in that process. Every figure, table, map, analysis, and example in this book was created using R. By building the entire book through code, we follow the same step-by-step process you will learn, one that anyone can run again and get the same result (this is called a reproducible workflow).

Rather than focusing on programming for its own sake, this book uses R as a tool to explore real Earth and environmental problems. The goal is not to memorize commands, but to develop a way of thinking: how to organize data, recognize patterns, ask meaningful questions, and interpret results in a scientific context.

The structure of the book reflects how real analysis works. We begin with the basics of R and gradually move toward working with real-world datasets. Along the way, we explore key ideas in data handling, spatial analysis, time series, and modeling. Each chapter builds on the previous one, using consistent examples so that concepts feel connected rather than isolated.

A central idea throughout the book is that simple methods, when used carefully, are often more powerful than complex techniques that are not fully understood. For this reason, the emphasis is on clarity, interpretation, and workflow. Every step is explained in a way that helps you understand not only what the code does, but why it matters.

Think Like an Earth Data Analyst

This book assumes no prior experience with R or programming. R is a free program you install on your computer to work with data, and we will show you exactly how to set it up in Chapter 1. If you are new to coding, you may feel uncertain at first. That is normal. The goal is not to learn everything at once, but to become comfortable reading code, running scripts, and gradually building confidence.

By the end of the book, you should be able to:

- read and understand basic R scripts,
- work with real geoscience datasets,
- visualize spatial and time-based patterns,
- build and interpret simple models,
- and approach new problems with a structured analytical workflow.

Most importantly, you should begin to think like a data analyst working across Earth and sustainability systems: asking questions, testing ideas, and interpreting results carefully and responsibly.

This book is not about getting perfect results. It is about learning how to ask better questions. Every question is worth asking; some answers simply need more thinking. Taking the time to understand a problem is often more important than trying to solve it immediately. Our goal is to give you real answers when possible, and when they are not, to show you how to approach them.

Acknowledgments

We would like to express our gratitude to the many students and colleagues who have shaped the way we think about teaching data analytics in Earth science.

Through years of teaching geostatistics and spatial analysis across universities in Europe and the United States, we have learned a simple but important lesson: students do not struggle with complexity; they struggle with abstraction. When concepts remain theoretical, they are easily forgotten. When they become hands-on, they become meaningful. This book was written with that philosophy in mind.

One recurring observation has been the limitation of "click-and-go" software. While such tools are easy to learn initially, they often become outdated as interfaces change and workflows evolve. Tutorials that rely on sequences of clicks rarely remain useful beyond a short period of time. In contrast, programming provides a more stable and transferable way of thinking. Functions, logic, and structure persist over time, even as tools evolve. For this reason, we strongly believe that coding is not just a technical skill, but a long-term investment in how students approach problem-solving in Earth and Sustainability Data Analytics.

We would like to thank Gail Bennington, adjunct faculty member and former director of the Hofstra University Summer Science Research Program (HUSSRP), for providing the opportunity to mentor high school students in research. Working with students who had no prior coding experience, yet successfully developed projects and presented their work at scientific conferences and competitions, reinforced a central belief behind this book: that with the right approach, anyone can learn to think analytically, work with data, and contribute meaningfully to research, regardless of their starting point.

We would like to acknowledge Charalambos Petrocheilos, whose introduction to R programming fundamentally influenced the way we approach data analysis. This experience reinforced a guiding principle that shaped this book: *think twice, code once.*

Finally, we are deeply grateful to our families for their patience and understanding during the many moments when this book required time beyond our professional responsibilities. Their support made this work possible.

How This Book Progresses

This book is designed as a practical introduction rather than a comprehensive reference manual. The emphasis is on understanding workflows, interpreting patterns, and developing analytical thinking through real Earth and sustainability applications. This book is organized as a gradual progression from understanding data to interpreting Earth systems analytically.

- Chapter 1 → learning R
- Chapter 2 → preparing trustworthy data
- Chapter 3 → understanding spatial structure
- Chapter 4 → understanding temporal structure
- Chapter 5 → understanding relationships, prediction, and an introduction to machine learning
- Chapter 6 → applying Earth and sustainability data analytics

Final Integration:

- Chapter 7 → understanding spatial and temporal global change and sustainability

Each chapter builds on the previous one, using consistent datasets and examples to create a connected learning experience. The goal is not to cover every possible method, but to develop a way of thinking that can be applied to new problems and datasets in the future. As you move through the book, the focus gradually shifts. In the early chapters, you may find yourself asking, "What does this function do?" Later, the more important question becomes, "Does this result make scientific sense?" Learning to evaluate results critically is one of the most important skills in Earth and Sustainability Data Analytics.

Each chapter follows a familiar learning rhythm, but the purpose is not repetition. The repeated structure is designed to help you stay oriented while the scientific reasoning becomes progressively more advanced. The book is also designed with different learning levels in mind. Beginners can successfully complete only the Part A sections in the earlier chapters and still build a strong foundation in Earth and Sustainability Data Analytics. Parts B and C gradually introduce more advanced workflows, spatial analysis, modeling concepts, and real-world applications.

Throughout this book, do not try to memorize every command. Focus on the questions the data are helping us answer. The code is only the means; scientific interpretation is the goal. By the end of the book, you should have a solid foundation in how to approach Earth and sustainability data analytically, with the tools and mindset to explore new questions on your own.

Part I

Foundations

Chapter 1

Introduction to R

Getting Started with RStudio and Basic R Concepts

1.1 Introduction

This chapter gives you a first look at the main ideas you will use in R. You do not need to remember everything right away. The goal is to help you feel comfortable opening RStudio, running code, and reading simple R scripts.

We will return to these ideas later in the book. You will see them repeatedly in different contexts, and they will become second nature with practice.

This chapter is organized into three parts:

- Part A introduces core concepts
- Part B introduces packages and libraries
- Part C provides optional topics for learners who want to explore further

In later chapters, you will use these same ideas to work with real Earth and sustainability data, including maps (space), time series (time), environmental systems, and human impacts.

1.2 Part A: Core Concepts

1.2.1 A Note for Windows and Mac Users

This book was written to work on both Windows and macOS. Nearly all code examples are identical across operating systems.

When keyboard shortcuts are shown, Windows users should use **Ctrl**, while Mac users should use **Command**. For example:

- **Ctrl + Enter** (Windows)
- **Command + Enter** (Mac)

Other differences between operating systems are minor and will not affect most examples in this book.

1.2.2 Before You Begin: Installing R and RStudio

Before opening RStudio, you need to install two free programs in this order:

Install **R** from https://www.r-project.org, then click "Download R" and choose your operating system. Install **RStudio** from https://posit.co, then click "Download RStudio Desktop (Free)".

Always open RStudio, not R directly. RStudio is the environment you will work in; R runs behind the scenes.

1.2.3 Getting Oriented in RStudio

RStudio panes

RStudio has different sections, but you only need to focus on two at the beginning:

- **Script** (top left) → where you write your code
- **Console** (bottom left) → where you see results

Getting Started: Your First Setup

> **i** Create a new script
>
> Press **Ctrl + N** to create a new script in the top-left section of RStudio.

Think of it like this: the script is your notebook (you write here), while the console is your calculator (you see answers here). There are more panes, but these two are the most important when you start. You will use the script to write and save your code, and the console to see the results of running that code. For reference (don't worry about memorizing this), RStudio is usually organized into four main areas:

- **Source pane (Script area)**: where you write and save your code
- **Console**: where R shows results after you run code
- **Environment / History**: shows what you created and what commands you previously ran in R.
- **Files / Plots / Packages / Help**: where you view files, graphics, installed packages, and documentation

Script versus console

Run code with Ctrl + Enter

For this course, always write your code in a script and run it with **Ctrl + Enter**. The console is useful for testing, but the script is where your work should be. Code typed in the **console** runs, but disappears when you close R. Think of the console like scratch paper: you can test things quickly, but they are not saved. A script is like your notebook because you can save your work and return to it later.

You can run a line by placing the cursor anywhere on that line; there is no need to keep the cursor at the beginning or end of each line. You can also select multiple lines and run them together.

> ⚠ Why you should use a script
>
> Writing code only in the console means you lose your work when you close R. Always use a script for anything you want to keep.

Before continuing, it helps to know a few basic actions in RStudio such as how to create, run a script, and ask for help.

Creating and saving a script

To create a new script:

- Click **File → New File → R Script**
- Save it with **Ctrl + S**

Saving your script helps you keep your work and return to it later.

Writing comments

Sometimes you want to write notes for yourself inside your code. These notes are called comments. You can write notes in your code using the `#` symbol, which starts a comment. Everything to the right of `#` on the same line is ignored by R. Comments are not run by R. They help explain what your code is doing. For example:

```
# The hash symbol (#) starts a comment.
# R ignores comments when it runs code.
```

```
x <- 10  # store the value 10 in the object x
x
```

```
[1] 10
```

The arrow `<-` means:

"put this value into this object."

When you run these lines with **Ctrl + Enter**, R calculates the answers and shows the results in the Console. Running the object name `x` by itself tells R:

"Show me what is stored inside `x`."

You may also see `=` used for assignment in some R code (for example, `x = 10`). Both work, but `<-` is the standard style in R and is what this book uses throughout.

> **!** How to run code in R
>
> R does nothing until you tell it to run code. Use **Ctrl + Enter** (Windows) or **Command + Enter** (Mac) to run code from your script.
>
> You can run code by:
>
> - placing your cursor anywhere on a line and pressing **Ctrl + Enter**
> - selecting multiple lines and pressing **Ctrl + Enter**
> - placing your cursor anywhere inside a multi-line command and pressing **Ctrl + Enter**
>
> You do **not** need to copy code into the Console.

Getting help

At the console, if you type:

```
?mean
```

R will open the documentation (help) page for `mean()` in the Help pane (bottom right), a built-in tool that averages numbers. Most help pages include a description, the inputs the tool accepts, and worked examples near the bottom. (We explain functions and their inputs in detail later.)

1.2.4 Your First Commands

Basic arithmetic

When you open RStudio for the first time, focus on two areas:

- Write your code in the Script pane and run it
- See the results in the Console

A typical workflow is to write code in the script, run it with **Ctrl + Enter**, and view the results in the Console or Plots pane. If you get an error in the console, read it carefully to understand what went wrong. Errors are normal when learning programming.

```
# Run by holding ctrl and pressing enter
# on each line
5 + 3
```

```
[1] 8
```

```
10 - 4
```

```
[1] 6
```

```
6 * 2
```

```
[1] 12
```

```
8 / 4
```

```
[1] 2
```

```
1:10
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

We explain the `:` (which makes a sequence of numbers) and the `[1]` marker in the output just below.

Spaces

- R ignores extra spaces in most situations, although consistent spacing makes code easier to read.

In R, the `:` symbol creates a sequence of numbers.

So:

```
1:10
```

means:

1, 2, 3, 4, ..., 10

This is a quick way to create consecutive values.

The `[1]` is added automatically by R. It shows the position of the first value in the output. You do not need to type it yourself. It becomes useful later when working with larger results.

> **!** When R gets "stuck"
>
> If you run an incomplete command such as:
>
> ```
> 5 +
> ```
>
> R console will show a `+` in the console and wait for you to finish the command. To stop this and return to the normal prompt (the `>` symbol where R waits for your next command), click the console and press **`Esc`** on your keyboard. This happens often when a parenthesis, quote, or operation is not completed.

1.2.5 What Is an Object?

An object is like a labeled box where you store a value. Objects help us save information so we can use it again later without retyping it. Almost everything in R is stored inside objects.

Store a value

You can create an object by assigning a value to it. For example, `x <- 10` means "store 10 in a box called x".

This saves a value so you can use it later:

```
x <- 10
```

Think of it like putting a label on a box:

- `x` is the label
- `10` is what's inside the box

You use `x` to save a value so you can use it again later. This is done with the symbol `<-`. You type `<-` by pressing the less-than `<` key followed by the hyphen `-` key. This allows you to reuse the value later without typing it again.

> **Why use objects?**
>
> Imagine you calculate a temperature value once and want to use it many times later. Instead of retyping the number every time, you can store it in an object and reuse it whenever you need it.

Create your first graph

Let's visualize numbers by creating a graph using two objects, `day` and `rainfall_mm`. We will create two short lists of numbers (these are called vectors, explained in the next section) and plot them against each other.

```
# Create two vectors
# of equal length
day <- 1:5
rainfall_mm <- c(0, 12, 5, 0, 8)
```

These code lines create five day numbers (1 to 5) and the rainfall in millimetres on each day, and store them in objects called `day` and `rainfall_mm`.

```
# Create a simple plot of day and rainfall
# The graph appears in the Plots pane (bottom right)
plot(day, rainfall_mm)
```

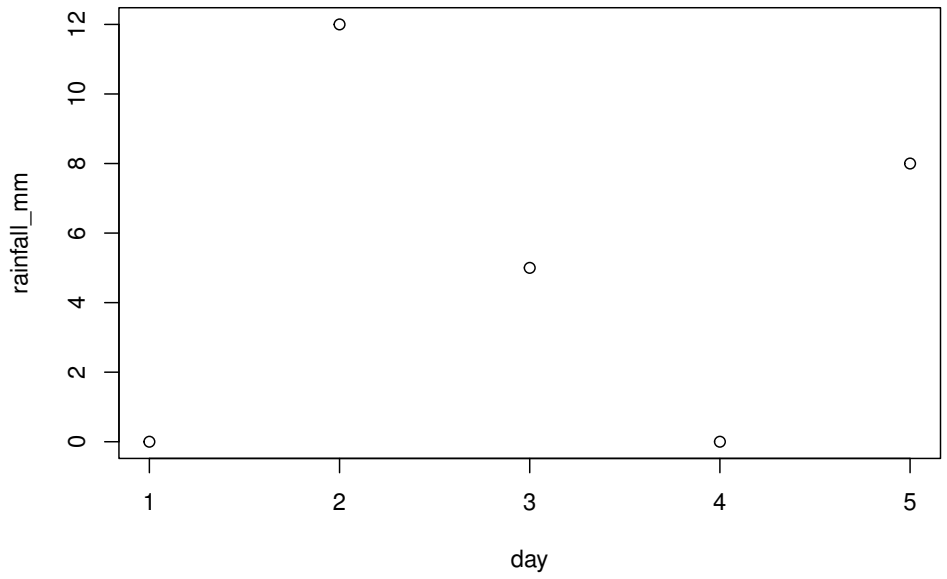


Figure 1.1: A simple scatter plot of daily rainfall (mm) over five days.

Because we gave R two vectors to plot:

- `day` becomes the horizontal axis
- `rainfall_mm` becomes the vertical axis

The rainfall changes from day to day, so the points do not form a straight line. Each dot represents one pair of values, such as `(1,0)`, `(2,12)`, `(3,5)`, and so on. The pair `(2,12)` means day=2 and rainfall=12, which places a dot above the second day at a height of 12. On days with no rain the value is 0, so the dot sits on the bottom axis.

Seeing data as a picture helps you notice patterns quickly. The rainfall rises and falls across the five days, with some days showing no rain at all (Figure 1.1). Even small datasets can reveal important structure when visualized. In geoscience, plots are one of the main ways we explore data.

1.2.6 Vectors

A vector is a group of values stored together. Usually, all values inside a vector should be similar, such as all numbers or all text. The `c()` command (short for "combine") joins several values into one vector.

```
temperature <- c(15.2, 16.1, 16.8, 17.4)
plot(temperature)
```

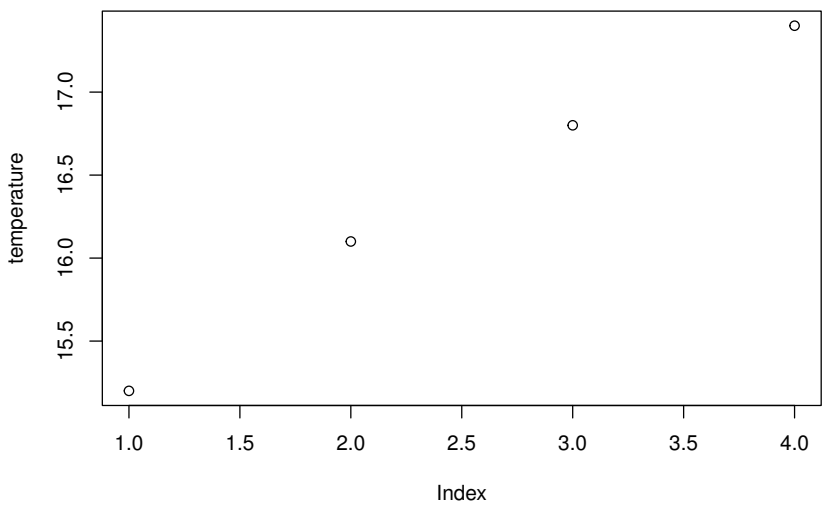


Figure 1.2: A simple plot of the four values in the temperature vector, shown against their position (index).

Vectors are commonly used in real environmental and sensor datasets because they store groups of related values together. In this example, the `temperature` vector contains four numeric values.

You can think of a vector as a list of numbers that belong together. When you create a vector, all values must be of the same type (e.g., all numbers or all text).

Plotting a single vector

Because we only gave R one vector, R automatically uses the positions of the values:
1, 2, 3, 4
for the horizontal axis.
So this plot shows:

- position on the x-axis
- temperature value on the y-axis

Common mistake: mixing numbers and text

A vector usually works best when it contains similar kinds of values.
For example:

```
c(1, 2, 3)       # all numbers
c("A", "B")      # all text
```

Mixing numbers and text together can lead to unexpected results.
For example:

```
c(5, "number")
```

If you ran this, the `5` would be shown in quotes as `"5"`, meaning R now treats it as text.
You can think of it like this:

- `5` is a real number that R can calculate with.
- `"5"` is more like a picture or label of the number 5.

It is similar to the difference between:

- a real burger you can eat
- and a picture of a burger

The picture looks like a burger, but you cannot eat it.
In the same way, `"5"` looks like a number, but R treats it as text instead of something to calculate with. This can cause problems later if you try to perform calculations with that vector.
Sometimes numbers should NOT be treated as numbers.
For example:

```
zipcode <- "11549"
```

A ZIP code looks numeric, but we usually do not calculate with it.
We would not add ZIP codes, multiply ZIP codes, or calculate averages of ZIP codes. A ZIP code is really a label, not a quantity. The same is true for:

- phone numbers, groundwater monitoring station IDs
- borehole IDs, earthquake event IDs
- satellite image IDs, sample numbers
- GPS station names, geological unit codes, and many other labels

Even though some of these contain numbers, they are often **labels rather than quantities**.
Always check that your vectors contain the type of data you expect.

An object can contain one value or many elements. Besides numbers and text, objects can also store vectors, data frames, lists, and arrays (different ways of organizing data that we introduce later in the chapter).

```
name <- "Earth" # This is a text object (character)
number <- 25    # This is a numeric object
flag <- TRUE    # This is a logical object (holds only TRUE or FALSE)
```

TRUE and FALSE are special values that represent yes/no or true/false situations.

Use `str()` to quickly check what an object contains. Sometimes you forget what kind of data an object contains. The `str()` function is like asking R:

"What is inside this object?"

Sometimes R gives errors because the data type is not what you expected.

For example:

- text instead of numbers,
- or TRUE/FALSE values instead of text.

```
str(name)
```

```
chr "Earth"
```

```
str(number)
```

```
num 25
```

```
str(flag)
```

```
logi TRUE
```

In this output, `chr` means character (text), `num` means number, and `logi` means logical (a TRUE or FALSE value).

The `length()` function tells us how many values are in the vector. This is useful when working with larger datasets because you often need to know how many values or observations you have.

```
length(temperature)
```

```
[1] 4
```

Before continuing, ask yourself

- What does the vector store?
- How many values does it contain?
- Which value is at position 1?

Indexing in vectors

You can pick specific values from a vector. The first value is at position 1, the second at position 2, and so on. The brackets `[ ]` are used to select values from an object.

Brackets `[ ]` are used to pull values out of an object. You can think of them like asking:

"Which position do I want to look at?"

2:4 means the values 2, 3, and 4.

```
temperature[1]   # first value
```

```
[1] 15.2
```

```
temperature[2:4] # positions 2 to 4
```

```
[1] 16.1 16.8 17.4
```

This means: "give me the first temperature value." Think of it like positions in a list (e.g. 1 = first item, 2 = second item). The second line says: "Give me values starting at position 2 and ending at position 4."

Notice the difference:

- `( )` are usually used with functions
- `[ ]` are used to access positions inside objects

Quick checkpoint

At this point, make sure you understand but not necessarily memorize:

- how to store values in an object,
- how to create a vector with `c()`,
- how to run code with **Ctrl + Enter**,
- and how to access values using brackets `[ ]`.

These are some of the most important foundations in R.

Combining values into a vector

We already used `c()` earlier to combine values into a vector. A vector is a group of values stored together in one object. You will use vectors often because they are one of the simplest ways to store data in R.

By the way, `c()` and `plot()` are examples of functions: small ready-made tools that do something for you (we cover functions properly in the next section). For example, `plot()` creates graphs and other functions perform calculations. A function starts with its name, followed by parentheses `()`. R keeps reading the function until it finds the closing parenthesis `)`.

This is why multi-line functions still work correctly. You can run a multi-line function by placing your cursor anywhere within the function and pressing **Ctrl + Enter**.

```
values <- c(2, # you can run this function
            4, # from any line within it
            6,
            8)
# This line displays the object while
# the previous lines create the vector
values
```

```
[1] 2 4 6 8
```

Did you notice that the `c()` function can be written across multiple lines? R keeps reading until it finds the closing parenthesis `)`. This allows you to organize your code in a way that is easier to read.

> ⚠ Avoid using reserved names
>
> Avoid naming objects as `data`, `list`, `mean`, or `plot`. These are existing R functions and can cause unexpected errors. Use names like: data_temp, list_values, or mean_value.

Try this yourself:

```
# Try this yourself
rain_mm <- c(5, 10, 15)
snow_mm <- c(2, 4, 6)

# Calculate total precipitation
# by adding the vectors
total_precip <- rain_mm + snow_mm
total_precip
```

```
[1]  7 14 21
```

Notice how we used descriptive names like rain_mm instead of just `a` or `x`. This is a great habit to build early. It makes your code much easier to read and reminds you of the units you are working with.

When we add them, R performs the operation element by element to calculate the total for each event. In other words, R adds the values one position at a time:

- first with first → $5 + 2 = 7$
- second with second → $10 + 4 = 14$
- third with third → $15 + 6 = 21$

Simple summary

These functions help you quickly understand your data. More advanced methods for summarizing data will be covered in later chapters, but these are good starting points for exploring vectors. The `summary()` function gives a quick overview of the data, including among other values the smallest value, the middle value, the average, and the largest value.

```
mean(values)
```

```
[1] 5
```

```
summary(values)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    2.0     3.5     5.0     5.0     6.5     8.0
```

The `1st Qu.` and `3rd Qu.` columns are called quartiles: `1st Qu.` is the value a quarter of the way through the sorted data, and `3rd Qu.` is the value three quarters of the way through. You do not need them for now.

> **i** Common beginner mistakes
>
> - Forgetting to run the code (nothing happens until you press **Ctrl + Enter**)
> - Forgetting to finish a function (for example, a missing closing parenthesis) or leaving a command incomplete (for example, `5 +`) can cause errors or unexpected behavior.
> - Running code only in the Console instead of saving it in a script (you lose your work when you close R)
> - Overwriting objects accidentally by reusing the same name

See how a previous value is lost:

```
x <- 10
x <- 5   # (10 replaced by 5)
```

- Using names that already belong to existing functions, such as `mean`, `plot`, or `data`

```
mean <- 10 # Do not run this; it is shown only to explain the mistake.
↪ It replaces the mean() function with the number 10, so restart R to
↪ restore the function.
```

- Forgetting parentheses in functions:

```
mean          # wrong
mean(values)  # correct
```

1.2.7 Functions

Using Functions

A function is a tool that does something for you. Do not worry about understanding every symbol yet.

You can **read this from the inside-out** (start with the part deepest in the parentheses, then work outward):

- `c(...)` first creates a vector
- `mean(...)` then calculates the average of that vector

```
# mean function with a vector of numbers as argument
  mean(c(1, 2, 3, 4, 5))
```

```
[1] 3
```

This function returns a single value based on the numbers you provide. You give the function input, and it returns output. The parentheses `()` are where you place the information you want the function to use. For example, `sqrt()` finds a square root, and `round()` rounds a number. The information (input) you give to a function is called an argument.

Functions are ready-made tools in R that take your input and produce a useful result.

Functions let you:

- calculate values
- create plots
- work with data
- and automate tasks

For example:

- `plot()` creates a graph
- `sqrt()` calculates a square root
- `round()` rounds a number to a specified number of decimal places

```
sqrt(16) # square root function
```

```
[1] 4
```

```
log(10)  # logarithm function
```

```
[1] 2.302585
```

```
round(3.14159, 2) # round to 2 decimal places
```

```
[1] 3.14
```

Creating your own Functions

A function has a name, parentheses (not brackets), and sometimes arguments. Let's create a simple function that adds two numbers together. This is just an example to show how to define a function in R. The function takes two arguments, `a` and `b`, and returns their sum.

```
add_two_numbers <- function(a, b) {
  a + b
}
```

And, let's call the `add_two_numbers` function with specific values to see how it works:

```
add_two_numbers(5, 10) # This will return 15
```

```
[1] 15
```

We can create a function to perform a specific task. For example, a function to convert Celsius to Fahrenheit. A function is defined with the word `function`, followed by its inputs (arguments) in parentheses and, in curly braces `{}`, the instructions the function carries out (this part is called the body). A function usually takes an input, performs an operation, and returns a result. By default, R shows the last line as the result. For example:

```
celsius_to_fahrenheit <- function(celsius) {
  (celsius * 9/5) + 32
}
# And you can call the function with a specific value:
celsius_to_fahrenheit(25) # converts 25°C to Fahrenheit, returns 77
```

```
[1] 77
```

Common mistakes when writing functions

- Not using curly braces `{}` to define the body of the function, which can lead to syntax errors
- Using the same name for the function and its argument, which can cause confusion and errors. (e.g. `celsius_to_fahrenheit <- function(celsius_to_fahrenheit) { ... }`) is not recommended

Before moving on

Make sure you understand these core ideas before continuing:

- how to create an object
- how to run code with **Ctrl + Enter**
- how to create a vector with `c()`
- how to use brackets `[ ]` to access values
- and how functions use parentheses `( )`

These ideas will appear repeatedly throughout the rest of the chapter.

Parentheses vs. Brackets

Parentheses `()` and brackets `[]` have different jobs in R.

- Parentheses `()` are used with functions. They tell R to perform an action.

- Brackets `[]` are used to pull values out of objects. They tell R which positions you want to access.

For example:

- `mean(values)` calls the `mean()` function
- `values[1]` accesses the first value in the vector

1.2.8 Data Frames

A data frame is a table made of rows and columns. You will use data frames often in R because many real datasets look like spreadsheets with rows and columns.

```
df <- data.frame(              # create a data frame
  station = c("A", "B", "C"), # station names column
  temp = c(15.2, 16.4, 14.8), # temperature values column
  rain = c(2.1, 0.0, 5.4)     # rainfall values column
)
```

A very common function to explore a data frame is `head()`, which shows the first few rows and all columns.

```
head(df)
```

```
  station temp rain
1       A 15.2  2.1
2       B 16.4  0.0
3       C 14.8  5.4
```

The `names()` function shows the column names.

```
names(df)
```

```
[1] "station" "temp"    "rain"
```

Use `str()` to quickly see what something contains. It shows what kind of values each column contains, such as numbers or text.

```
str(df)
```

```
'data.frame':   3 obs. of  3 variables:
 $ station: chr  "A" "B" "C"
 $ temp   : num  15.2 16.4 14.8
 $ rain   : num  2.1 0 5.4
```

Here `3 obs. of 3 variables` means 3 rows and 3 columns, and each `$ name:` line is one column with its data type (the trailing `...` just means more values follow). You do not need to understand all of this yet. We will use it again when working with real data. These functions help you quickly understand the data content before working with it. Let's access a column in the data frame using a name:

```
df$temp
```

```
[1] 15.2 16.4 14.8
```

The `$` symbol means: "Look inside this table and give me this column."

> **Tab**: one of your most useful tools in R
>
> By typing `df$` and hitting `tab`, you can see all the column names available in the data frame. Pressing `Tab` again can autocomplete the column name instead of typing it manually. This is a helpful way to avoid typos and remember the structure of your data. You can also access a column by position (e.g. `df[ , 2]` for the second column), but using names is clearer and less error-prone.

1.2.9 Indexing in data frames

You can access a row by position in your data frame by using brackets [ ] (not parentheses):

```
df[1, ]
```

```
  station temp rain
1       A 15.2  2.1
```

Inside the brackets: - the first position is the row - the second position is the column

Just like in vectors, the first row is at position 1, the second row is at position 2, and so on. The comma `,` separates rows and columns. So `df[1, ]` means:

"give me the first row and all columns".

Also, you can access a specific value by row and column position of your data frame:

```
df[1, 2]
```

```
[1] 15.2
```

This means:

- row 1
- column 2

> **Data frames are like spreadsheets**
>
> Each column of a data frame can have a different type of data (e.g. one column can be numeric, another can be text). This makes data frames very flexible for storing tabular data. However, all values within a single column must be of the same type (e.g. all numbers or all text). Also, all columns must have the same number of rows. This is a limitation that lists and arrays do not have.

So far we have worked with simple objects. Some real-world datasets are too complex to fit into a simple spreadsheet-style table. Many Earth datasets contain measurements that vary across location, depth, or time. Lists and arrays help store this type of data.

> **You already know more than you think**
>
> At this point, you already know how to:
>
> - create objects,
> - store vectors,
> - run functions,
> - create simple plots,
> - access values by position,
> - and work with basic tables in R.
>
> These are the same core ideas used in real data analysis workflows.
> You are not expected to memorize every command. The goal is to become comfortable reading code, testing ideas, and understanding how data are organized.
> Everything from this point forward will build on these same foundations.

1.3 Part B: Packages and Libraries

R includes many built-in functions, but it can do much more through **packages**. A package is a group of extra functions you can add to R. Packages can also include datasets and tools that extend what R can do.

A simple way to think about it is:

- **R base** gives you the core tools
- **packages** give you extra toolboxes
- **functions** are the tools inside those toolboxes

At this stage, you do not need packages for every task. Most examples in this chapter use only base R functions. This is intentional, so you can first understand the core ideas: objects, functions, vectors, data frames, arrays, and loops. As you begin working with real datasets, you will start using packages. Packages add new tools to R for specific tasks. They are essential for working with real Earth and sustainability data. Many packages are created by other users and researchers. For example, the `maps` package allows us to draw geographic boundaries, such as countries and states. In later chapters, we will use more advanced packages to work with real-world data, including spatial and time series datasets.

You need to install a package only once with `install.packages("your_package")`, but you need to load it with `library(your_package)` every time you start a new R session.

```
# Install only once if needed.
# Uncomment this line the first time you use the package.
# install.packages("maps")

# Load the maps package
# Installed packages must be loaded each time you start a new R session
library(maps)
```

Common mistakes

It is easy to forget to load a package with `library()` before using functions from that package. Even if a package is already installed, R does not automatically load it when RStudio starts. Packages remain installed on your computer, but they must be loaded into the current R session each time you open RStudio or start a new session.

Another common mistake is confusing `install.packages()` with `library()`:

- `install.packages("package")` installs the package onto your computer and usually only needs to be done once.
- `library(package)` loads the package into the current R session and must be run whenever you start a new R session and want to use that package.

> Also remember that `install.packages()` uses quotes around the package name, while `library()` does not.

Accessing all functions in a package You can also call a function directly from a package when needed. This is helpful when you want to be very clear about where a function comes from. The double colon `::` means "from this package, use this function": `maps::map()` reads as "use the `map()` function from the `maps` package," even without loading the whole package first. This matters later when two packages share a function name and you want to specify which one to use.

You can also explore functions inside a package by typing the package name followed by `::` and then pressing **Tab**.

For example:

```
maps::
```

Pressing `Tab` will show available functions inside the maps package. This is useful when you do not remember the exact function name or want to explore what a package contains.

The next example draws U.S. state boundaries by calling a function directly from the maps package (Figure 1.3).

```
# Example of calling a package function directly
maps::map("state")
title(main = "Map of the United States")
```

Figure 1.3: Map of the United States generated using the maps package.

The `maps` package includes several built-in map datasets, such as `state`, `county`, `world`, `world2`, and `usa`. You can see the full list with `help(package = "maps")` or `library(help = "maps")`, which open the package documentation in the Help pane (bottom right).

- "state" → U.S. states
- "world" → countries around the world

If you want to plot a country such as Greece, use the built-in world map and specify the country name with the `regions` argument:

```
# Region names must be written in lowercase and in English.
map("world", regions = "greece", fill = TRUE, col = "lightblue")
```

Figure 1.4: Map of Greece generated using the maps package.

You can also plot the entire world map with `map("world")` to see all countries. The `regions` argument allows you to specify which country or region to display (Figure 1.4). The `fill = TRUE` argument fills the area of the specified region with color, and `col = "lightblue"` sets the fill color. The map is simplified and does not include labels. It is not meant for precise identification, but for basic visualization. You can customize the appearance of the map by changing these arguments to include more details (north, scale, legend etc).

Or, you can zoom in on a specific region, such as New York State (Figure 1.5).

```
map("state", regions = "new york")
title(main = "New York State")
```

Figure 1.5: Map of New York State generated using the maps package.

This example shows how we can select a region by name. In later chapters, we will work with spatial data where regions are selected based on their properties and geometry.

> **Region names must match the dataset**
>
> Region names in the `maps` package must be written in lowercase and must match the dataset naming. For example, "New Jersey" should be written as "new jersey", as shown in Figure 1.6. If you use a name that does not exist in the dataset, you will get an error.

To check more state names use:

```
map("state", plot = FALSE)$names
```

We set `plot = FALSE` because we want the state names without drawing a map. The `$names` part extracts the state names from the result returned by `map()`. This prints a long character vector of valid region names, beginning `[1] "alabama" "arizona" ...`, and you can use these names to plot specific states.

```
map("state", regions = "new jersey", fill = TRUE, col = "lightblue")
title(main = "New Jersey State")
```

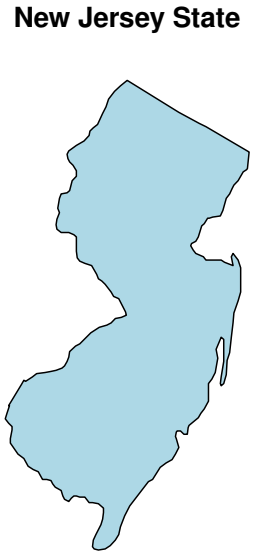


Figure 1.6: Map of New Jersey generated using the maps package.

1.3.1 A preview: interactive maps in R

Geoscience often involves working with maps. R can also create interactive, web-based maps with only a few lines of code.

> **i** A preview of what R can do
>
> The goal is not to memorize these commands. The goal is to see that R can create interactive maps and to understand that maps are another way to explore data step by step.

```
# Install only once if needed.
# Uncomment this line the first time you use the package.
# install.packages("leaflet")
library(leaflet)

m <- leaflet()   # create an empty map object
m <- addTiles(m) # add background map
m <- setView(    # set the initial view of the map
  m,
  lng = -73.6, # longitude
```

```
  lat = 40.8,  # latitude
  zoom = 8     # initial zoom level
)
m              # display the map
```

Figure 1.7: Interactive map demonstrating how R can create zoomable, web-based maps for exploring geographic information.

> **Longitude first, latitude second**
>
> `lng` means longitude and `lat` means latitude. Together they specify the location where the map initially opens. If they are accidentally switched, the map may open in the ocean or in a completely different location.

When you run this code in RStudio, the map appears in the Viewer pane and can be zoomed and panned interactively. In this printed version of the book, it appears as a static screenshot. Interactive maps are widely used in research websites, dashboards, sustainability projects, and science communication. We will not focus on interactive mapping in this book, but it is useful to know that R can create web-based maps with only a few lines of code.

In Chapter 3, we will return to mapping and begin working with spatial data more directly.

1.4 Part C: Beyond the Basics (Optional)

1.4.1 Preview: Lists and Arrays

Some data do not fit into simple tables. In these cases, we use lists and arrays to organize them. This section is optional, but it will help you understand more complex Earth datasets later.

Lists and arrays become essential when working with real Earth and sustainability datasets. You do not need to master lists and arrays yet. For now, just know that:

- a **list** can hold different kinds of objects
- an **array** can hold values in more than two dimensions

We will return to these ideas later when they become useful.

Lists and Arrays vs Data frames

Data frames organize data in rows and columns, which works well for tabular information like station measurements. But many Earth datasets include an extra dimension. For example, temperature can change with ocean location (`latitude` and `longitude`) and `depth`. Another example is `ice thickness` which can vary across `location` and `time`. Satellite data are often stored as grids (x,y) that repeat over time. These types of data cannot be easily stored in a simple table. Instead, we use lists and arrays to represent them.

1.4.2 Lists

A list is a container that can hold different kinds of data. For now, just think of a list as a box that can hold different items. A list can contain different types of data, such as numbers, text, vectors, or data frames. This is useful for storing complex data structures that do not fit neatly into a table. For example, you might have a list that contains a vector of numbers, a vector of text labels, and a data frame with measurements. Each element of the list can be accessed by its name or position.

Indexing in Lists

```
my_list <- list(        # an object with different types of data
  numbers = c(1, 2, 3), # three elements vector
  label = c("model_A", "model_B"),    # two text elements vector
  table = df            # a data frame element
)
my_list
```

```
$numbers
[1] 1 2 3
```

```
$label
[1] "model_A" "model_B"

$table
  station temp rain
1       A 15.2  2.1
2       B 16.4  0.0
3       C 14.8  5.4
```

To read this output, each `$name` heading marks one item in the list: `$numbers`, `$label`, and `$table`. Under `$table` you see the whole data frame printed out.

You can access parts of a list by name, using `$` followed by the element name:

```
my_list$numbers # access the numbers vector
```

```
[1] 1 2 3
```

```
my_list$label   # access the character (label) vector
```

```
[1] "model_A" "model_B"
```

You can also access them by position. For lists we use double brackets `[[ ]]`, which pull out the actual item stored at that position:

```
my_list[[1]] # access the first element (numbers vector)
```

```
[1] 1 2 3
```

```
my_list[[2]] # access the second element (label vector)
```

```
[1] "model_A" "model_B"
```

Access the second row of the data frame stored in the list (station "B" and its temperature and rainfall values):

```
# first [[3]] pulls out the data frame, then [2,] takes its second row
my_list[[3]][2,]
```

```
  station temp rain
2       B 16.4    0
```

Lists are very flexible and can be used to store complex data structures, such as the output of a model that includes multiple variables, parameters, and results. They are also useful for organizing data that have different types or lengths, such as a list of station measurements where each station has a different number of observations.

The next examples introduce ideas that are useful later in the book, especially for spatial and time-based Earth data. Focus on the main idea: some datasets contain multiple layers that change across space and time. Lists and arrays help us organize and work with these complex datasets.

1.4.3 Arrays

The next examples are more advanced than the earlier sections. Do not worry about memorizing the syntax.

An array is like a stack of tables, maps, or images.

Each table can represent:

- a different day
- a different depth
- or a different satellite image

Arrays store data in more than two dimensions. For example, an array can store satellite images collected over many days. You can think of an array like a stack of images.

- rows run from top to bottom
- columns run from left to right across the table
- layers are separate images stacked together

Here `1:24` makes the numbers 1 through 24, exactly enough to fill the array, since $3 \times 4 \times 2 = 24$.

```
arr <- array(1:24, dim = c(3, 4, 2))
arr
```

```
, , 1

     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12

, , 2

     [,1] [,2] [,3] [,4]
[1,]   13   16   19   22
[2,]   14   17   20   23
```

```
[3,]   15   18   21   24
```

In this output, the `, , 1` header marks layer 1 and the `, , 2` header marks layer 2. Each layer is one 3x4 table.

The dimensions `c(3, 4, 2)` mean:

- 3 rows
- 4 columns
- 2 layers

So this array contains two separate 3×4 tables stacked together.

> **How the array is stored**
>
> R fills arrays column by column rather than row by row.
> For example, the values 1, 2, and 3 are placed in the first column before R moves to the second column.
> The printed array in the Console and the output from `image()` may not appear identical because `image()` displays the values as a spatial grid rather than as a spreadsheet-style table.

Visualizing array layers

To access a value, use:

`arr[row, column, layer]`

Each layer of the array can be visualized as an image.

```
arr[1, 1, 1]  # row 1, column 1, layer 1
arr[1, 1, 2]  # same position, but in layer 2
```

The printed output above shows the two layers separately:

- `, , 1` represents the first layer (the first 3×4 table)
- `, , 2` represents the second layer (the second 3×4 table)

Each layer contains one 3×4 grid of values.

If you leave something empty before a comma, R understands that you are not selecting a specific row or column.

The empty spaces before the commas mean:

- "use all rows"
- "use all columns"

So this command says:

"show all rows and columns from layer 1."

```
# the image() function draws values as a colored grid; this shows layer 1
image(arr[ , , 1])
```

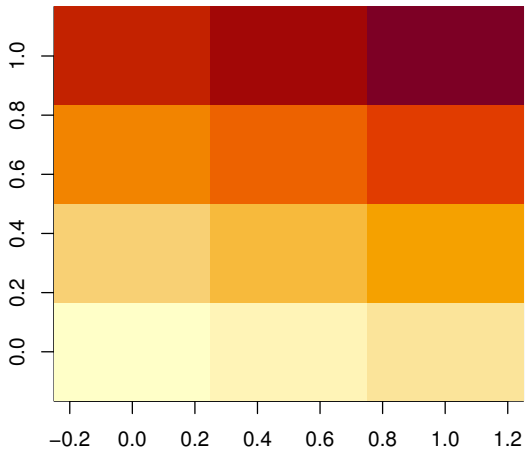


Figure 1.8: First layer of a 3D array visualized as an image. Light colors represent lower values and dark colors represent higher values.

The colored grid does not line up cell-for-cell with the printed table below, because `image()` rotates the values when it draws them.

```
arr[ , , 1] # print the first layer as numbers
```

```
     [,1] [,2] [,3] [,4]
[1,]    1    4    7   10
[2,]    2    5    8   11
[3,]    3    6    9   12
```

The image shows how values are distributed across the grid. The printed values give exact numbers, while Figure 1.8 provides a visual summary of their spatial distribution.

Each cell in the image represents one value in the array. The color shows the size of the value:

- lighter colors = smaller values
- darker colors = larger values

This helps you quickly notice differences across the layer.

Comparing the printed layer with the image helps you understand how values map to colors.

How the array is displayed

When R prints an array in the console, you read it like a table or spreadsheet. However, when you display it with `image()`, R treats the rows and columns as positions in space rather than as a spreadsheet.
As a result, the image may not look exactly like the printed table. Do not worry if the visual appears rotated or flipped compared with the console output. You did not make a mistake. This happens because `image()` draws rows from bottom to top (like a geographic map), while the console prints rows from top to bottom (like a spreadsheet). The values are exactly the same; only the orientation differs.

Subtracting images

You can also perform calculations between layers. For example, subtracting one layer from another can show how values change between time steps or conditions. This helps show how data change over time, such as calculating anomalies or differences between time steps. The resulting image shows where values increased or decreased between the two layers, which can represent change over time.

```
# subtract the first layer from
# the second layer and visualize the result
image(arr[ , , 2] - arr[ , , 1])
```

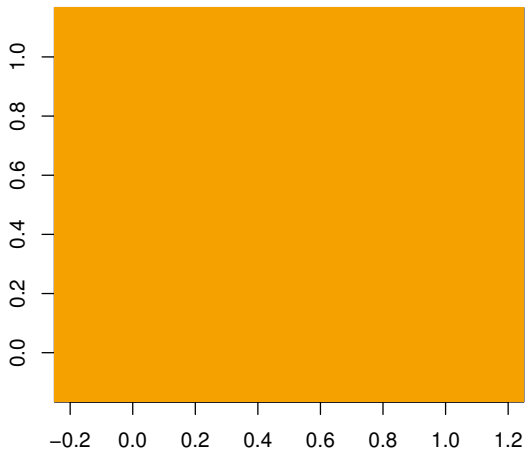


Figure 1.9: Difference between the second and first layers of the array. The uniform color indicates that every cell has the same value (12).

All values are equal (12), which results in a uniform image (Figure 1.9). The image appears as a single color because every cell has the same difference. This happens because the second layer contains values 13 to 24, while the first layer contains values 1 to 12, so each corresponding position differs by exactly 12.

Working with Multiple Layers

So far, we visualized each layer manually. But what if we had many layers (for example, 100 time steps)?

Instead of writing the same code repeatedly, we can use a for loop to automate the process. This allows us to visualize all layers in a stack without having to write separate code for each one. This is especially useful when working with large datasets, such as time series of satellite images, where you might have dozens or hundreds of layers to visualize.

1.4.4 Loops

A loop repeats the same action several times automatically.

Think of it like telling R: "Do this again and again."

Instead of writing the same code again and again, we use a loop. They are useful when the same operation must be done for many values.

Imagine you had 100 images instead of 2. Writing the same plotting command 100 times would be slow and tiring. A loop lets R repeat the same task automatically.

Everything inside the curly braces `{}` is repeated.

```
for (i in 1:5) {
  print(i)
}
```

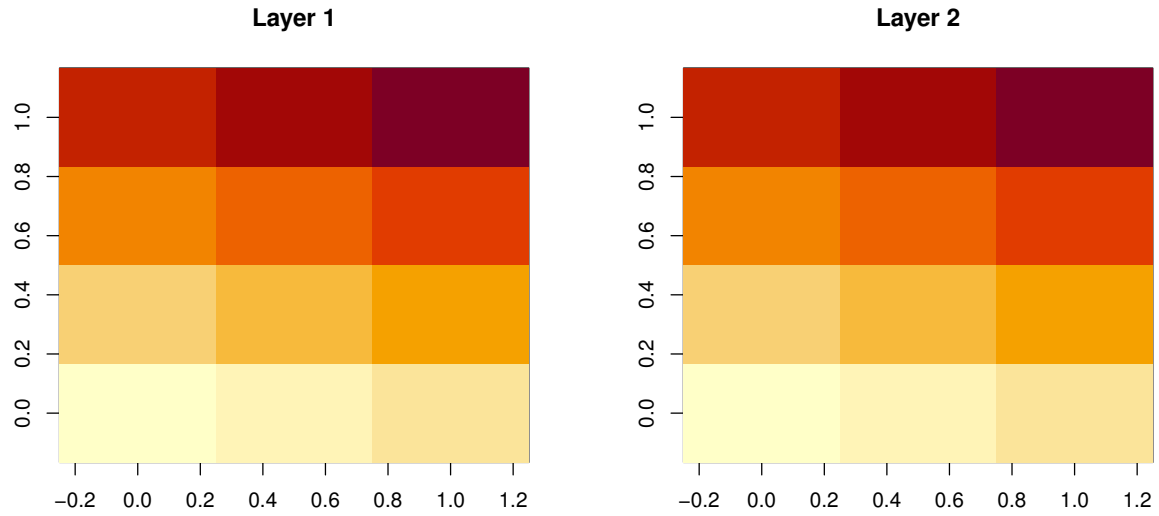
```
[1] 1
[1] 2
[1] 3
[1] 4
[1] 5
```

In this example, the object `i` takes the values 1, 2, 3, 4, and 5.

You can think of `i` as a number that changes automatically during the loop. The loop runs the code inside the curly braces `{}` for each value of `i`. So it will print the numbers 1 through 5, one at a time. Loops will be useful later when working with time series and spatial data.

A loop repeats the same action for different values. Let's use a loop to visualize both layers of our array without writing separate code for each layer:

```
# add a title to each plot
# indicating the layer number (1 or 2)
for (i in 1:2) {
  image(arr[ , , i])
  # paste() joins the word "Layer" with the number i
  # to make the title text
  title(main = paste("Layer", i))
}
```



(a) Array layer 1.

(b) Array layer 2.

Figure 1.10: The two array layers drawn by the loop.

Arrays and loops are useful when working with data that change across space and time, such as ice thickness or temperature grids. The loop runs through each layer of the array, that is, `i` takes the values 1 and 2. For each value of `i`, we plot the corresponding layer. This avoids repeating the same code multiple times.

These ideas will become important when working with real datasets, such as ice thickness changing over space and time.

Curly braces define the loop

A common mistake is forgetting the curly braces { }. These show which lines belong to the loop.

> In RStudio, enabling *Rainbow Parentheses* in Global Options can make matching parentheses, brackets, and braces easier to see.

1.4.5 A Volcano Temperature Application

Now let's use a small example to connect the ideas of arrays, indexing, and plotting.

We will use the built-in `volcano` dataset as a grid of values. For this example, we will treat those values as temperatures measured across a volcanic area.

Day 1 shows the original temperature field. Day 2 shows the same field with a localized warm area in the northern part of the map. We will store these two days in a three-dimensional array, where the first two dimensions represent space and the third dimension represents time.

- Day 1 = the original temperature field
- Day 2 = the same temperature field, but with a localized warm area in the northern part of the map

A 3D array is like a stack of maps. Each layer in the stack represents one day.

Here we modify a small square in the northern part of the grid to simulate localized heating, like a fumarole (a hot vent on a volcano). This creates a clear contrast with the surrounding area, making the change easier to visualize. The values `48:50` select a small square region in the northern part of the grid. This shows how one part of the data can change while the rest stays the same, which is common in Earth datasets.

```
# Create a 3D array sized to match the volcano grid
# (rows, columns, and 2 time layers).
# NA marks an "empty"/missing value (we fill it in below).
# nrow() and ncol() count the rows and columns of volcano.
temp <- array(NA, c(nrow(volcano), ncol(volcano), 2))

# Day 1: baseline temperature
temp[,,1] <- volcano

# Day 2: localized heating in the northern part of the map
temp[,,2] <- volcano

# Add heat to the volcano area on day 2
# Simulate localized warming
# like a fumarole (hot volcanic vent)
temp[48:50, 48:50, 2] <- temp[48:50, 48:50, 2] + 50
```

What this code does

- `temp` stores the full dataset.
- The first layer is Day 1.
- The second layer is Day 2.
- The line with `48:50, 48:50, 2` changes only a small area in the northern part of the map on the second day.

Think of it like warming only a small area in the northern part of the map while leaving the rest unchanged. Many Earth datasets change across both space and time, such as temperature, elevation, or satellite observations. The next example shows how to visualize those changes side by side.

```
# par() changes global plot settings
# and mfrow = c(1, 2) arranges two plots
# side by side in one row and two columns
par(mfrow = c(1, 2))
# Day 1
# col = terrain.colors(20) picks 20 map-like colors;
# axes = FALSE hides the axis numbers
image(temp[,,1], col = terrain.colors(20), axes = FALSE, main = "Day 1")
# add contour lines showing locations
# with the same temperature (called isotherms)
contour(temp[,,1], add = TRUE)
# Day 2
image(temp[,,2], col = terrain.colors(20), axes = FALSE, main = "Day 2")
contour(temp[,,2], add = TRUE)
# Restore the default plotting layout for future plots
par(mfrow = c(1, 1))
```

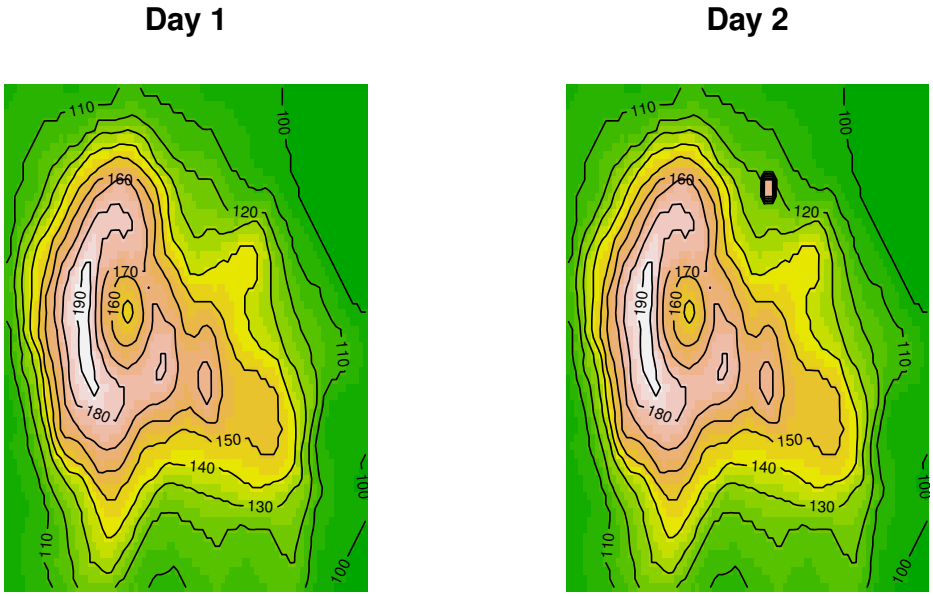


Figure 1.11: Simulated temperature over a volcano for two consecutive days. The second day shows increased temperature in a localized area of the northern part of the map, illustrating how spatial data can change over time.

Contour lines connect areas with similar values. In this example, they connect locations with similar temperatures, much like elevation contours connect locations with similar elevations on a topographic map. Every point along the same contour line has the same temperature.

Plot the two days side by side

In this example, we treat the `volcano` dataset as a temperature field. Each value represents the temperature at a specific location on the surface.

The data are stored in a three-dimensional array, where the first two dimensions represent space (rows and columns), and the third dimension represents time. Here, the first layer corresponds to Day 1, and the second layer corresponds to Day 2.

On Day 2, we increase the temperature in a small area of the northern part of the map to simulate localized heating. This shows a common pattern in Earth data: one area can change over time while nearby areas remain similar.

> **i** Why visualization matters
>
> Scientists use plots to notice patterns that are difficult to see in raw numbers.

1.4.6 Reading and Interpreting Code

In a script, you read code from top to bottom, and you need to understand what each line does. Read code one line at a time. After each line, ask: "What changed?" Each line builds on the previous one. New lines of code can change objects created earlier, and functions may be called that transform data. The result from one line is often used again in the next line. This is why it's important to read code carefully and understand the structure of objects and functions.

Here is a short example:

```
result <- mean(c(10, 12, 14, 16))
result
```

```
[1] 13
```

Read the code from the inside out and ask yourself:

1. What object (such as a vector, list, or data frame) is being created or modified? (Here: a vector of four numbers.)
2. What function (e.g. mean, for loop) is being used? (Here: `mean()`.)
3. What data are being selected or transformed?
4. What result should appear?

Let's break it down from the inside out:

- `c(10, 12, 14, 16)` creates a vector containing four numbers.
- `mean(...)` calculates the average of those numbers.
- `result <-` stores the average in an object called `result`.
- `result` displays the value stored in the object, which is the average of the numbers: 13.

> **Focus on the ideas**
>
> You are not expected to memorize everything in this chapter.
> The goal is to:
>
> - become comfortable seeing code,
> - understand the basic ideas,
> - and recognize common patterns.
>
> These concepts will appear many times later in simpler and more practical examples.

1.5 Exercises

Try the following exercises using the Earth-data concepts we just covered:

1. Create a vector called `temperatures` that contains three temperature values (in °C).
2. Calculate the mean temperature.
3. Create a data frame with three columns: `station`, `temp`, and `rain` (similar to the data-frame example earlier in this chapter).
4. Extract the first row of this data frame.
5. Write a `for` loop that prints the numbers 1 to 3.

1.6 Key Takeaways

- R works through objects and functions.
- Scripts are better than console-only work because they can be saved and reused.
- Vectors, data frames, lists, and arrays are data structures.
- Reading code carefully is just as important as writing code.
- These ideas will be used again when working with spatial and time-based data.

So far, we have focused on the basic tools of R: objects, vectors, functions, and data frames. These tools allow us to write code and understand how R works. But real analysis in Earth science does not begin with clean, simple examples; it begins with real data, often complex, incomplete, and shaped by natural processes.

The goal of this chapter was not to memorize commands, but to understand how R organizes and processes data. These ideas become meaningful when we apply them to real Earth datasets, where patterns represent physical processes such as climate variability, groundwater movement, or tectonic change.

In the next chapter, we take that step. You will begin working with actual Earth and environmental data from the United States Geological Survey (USGS): reading it into R, inspecting its structure, and preparing it for analysis. This step is important because good analysis begins with understanding your data. The quality of any result, whether a map, a time series, or a model, depends on how well the data are understood and organized before analysis begins.

R is not the goal. It is one of many tools used in Earth data analytics. The goal is to develop the ability to think critically about data, recognize patterns, and connect those patterns to real-world Earth, environmental, and sustainability processes. You are not expected to master computer science concepts. Instead, you are building practical skills that allow you to work with data, ask meaningful questions, and interpret results in an Earth science context.

1.6.1 References (Data and Further Reading)

1. R Project for Statistical Computing. https://www.r-project.org
2. RStudio / Posit. https://posit.co

3. Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. *R for Data Science*. https://r4ds.hadley.nz

4. maps package documentation. https://cran.r-project.org/package=maps